04 · Do objeto Kotlin aos bytes no socket

Camadas chamadas pelos coordenadores ProxyHandshake / ServerHandshake

ENVIO · do significado para a rede

1. DTO: ClientHelloPayload / ServerHelloPayload / ClientFinishedPayload + ProposedCryptoConfig
   Guarda campos com significado; ainda não é a trama.
2. HandshakePayloadCodec.encode...()
   DataOutputStream → ByteArrayOutputStream
   writeLong / writeInt / writeText / writeFixed / writeSized; writeConfig() só envia dados públicos.
3. HandshakeKeySchedule + HandshakeCrypto
   Type 1: HMAC(header || body).
   Types 2/3: encrypt(plaintext, key, iv, AAD).
4. HandshakeWire.frame() → ProtocolMessage
   MessageHeader + payload final (ByteArray).
   O header final é preparado antes de HMAC/AAD.
5. HandshakeWire.send(output, frame, limite)
   → ProtocolMessageCodec.write()
   → encodeHeader() → write(header) + write(payload)
   → flush() no OutputStream do socket.

RECEÇÃO · da rede para o significado

1. InputStream do socket
   TCP entrega um fluxo: uma leitura pode ser parcial.
2. ProtocolMessageCodec.read(input, limite)
   Lê 9 bytes de header; extrai tipo e tamanho.
   Valida tamanho antes de alocar; lê payload todo.
3. HandshakeWire.read(input, esperado, limite)
   Valida versão/release e Type esperado.
   Type 255 recebido → aborta.
4. O coordenador aplica autenticação / cifra
   Type 1: decode limitado → verificar HMAC.
   Types 2/3: decrypt() verifica a tag GCM
   antes de disponibilizar o plaintext ao decoder.
5. HandshakePayloadCodec.decode...()
   DataInputStream: readLong / readInt / readText / readFixed / readSized + readConfig().
6. Coordenador verifica PCC/CCC, sequências, desafio e DH conforme a etapa.

TCP · bytes ordenados; não preserva fronteiras de mensagens

TRAMA NO FIO · big endian

version: 2 bytes | release: 2 bytes | type: 1 byte | payloadLength: 4 bytes | payload: N bytes
Header = 9 bytes. payloadLength inclui prefixos públicos, ciphertext e tags, quando existirem.
Os DTOs, a PSK e os ficheiros XML não são serializados automaticamente para a rede.

SE ALGO FALHAR · localizar a camada primeiro

XML mal formado / tag em falta → CryptoConfigLoader.load(); nem chega a abrir o socket.
Combinação / perfil não suportado → CryptoConfigValidator / HandshakeConfigValidator.
Recusa de ligação / timeout → socket.connect(), accept() ou leitura com soTimeout.
Trama truncada / tamanho inválido → ProtocolMessageCodec.read(); tipo/versão → HandshakeWire.read().
HMAC inválido (Type 1) → PSK, chave derivada ou header/body diferentes.
Tag GCM inválida (Types 2/3) → chave, IV, AAD ou ciphertext diferentes / adulterados.
Tag válida mas sequência / CCC / desafio incorretos → verificações do coordenador.
catch: HandshakeWire.alert() tenta enviar Type 255; relança erro; Demo.use fecha a ligação.

Type 255 antes da sessão é diagnóstico não autenticado; não autoriza baixar a segurança.
Este mapa descreve o código atual de 26/10/2026; atualizar ao integrar o streaming protegido.